

Rapport d'évaluation du modèle de sentiment — Sentiscope

Étudiant 4 — Évaluation, métriques & rapport

1. Contexte et méthodologie

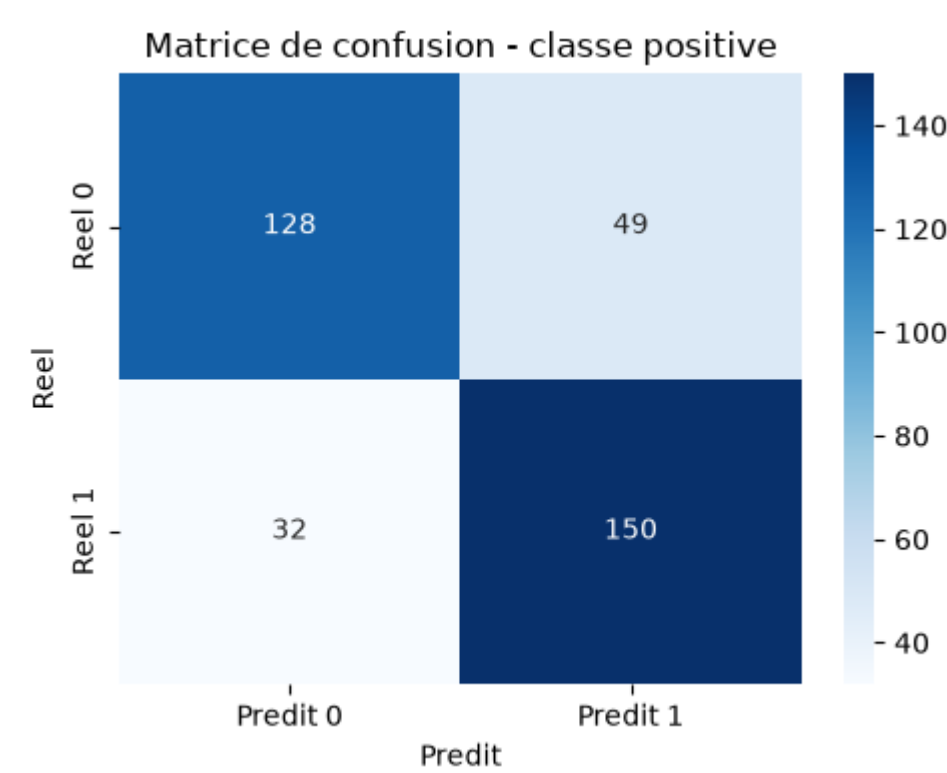
Le modèle évalué est une régression logistique sur représentation TF-IDF (`model.py`, `SentimentModel`), qui produit un score continu dans $[-1, 1]$ pour un tweet donné (-1 = très négatif, 1 = très positif). Ce score est binarisé à un seuil de 0 pour produire les deux labels attendus par le schéma de la table `tweets` : `positive` (`score > 0`) et `negative` (`score < 0`).

Entraînement : 50 000 tweets réels, échantillon équilibré ($\approx 50/50$) tiré du jeu public [Sentiment140](#).

Validation : les 359 tweets du jeu de test *annoté manuellement* de Sentiment140 (182 positifs, 177 négatifs), un fichier distinct du corpus d'entraînement — jamais vu par le modèle pendant l'apprentissage. C'est un point méthodologique important : évaluer sur des données déjà utilisées à l'entraînement aurait gonflé artificiellement les résultats.

2. Matrices de confusion

2.1 Classe positive

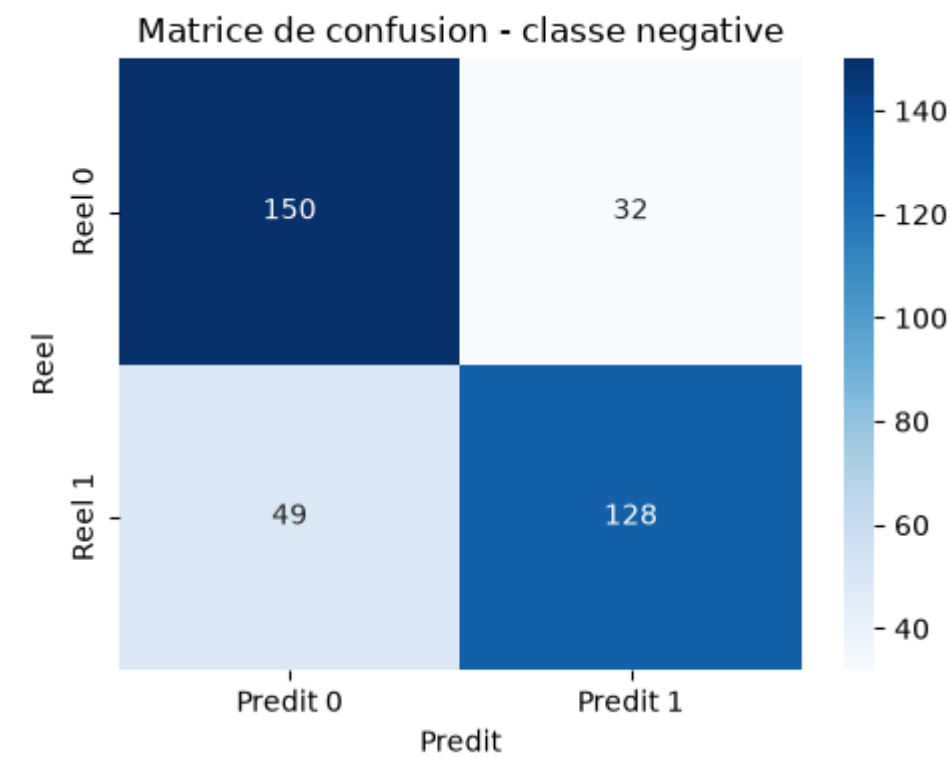


Réel \ Prédit	Prédit non-positif	Prédit positif
Réel non-positif	128 (VN)	49 (FP)
Réel positif	32 (FN)	150 (VP)

Support : 177 tweets réellement non-positifs, 182 tweets réellement positifs.

Interprétation : le modèle retrouve 150 des 182 tweets réellement positifs (rappel 82 %), mais 49 tweets réellement négatifs sont classés à tort comme positifs. Ces faux positifs sont la principale source d'erreur sur cette classe (voir §4.2 pour des exemples concrets).

2.2 Classe négative



Réel \ Prédit	Prédit non-négatif	Prédit négatif
Réel non-négatif	150 (VN)	32 (FP)
Réel négatif	49 (FN)	128 (VP)

Support : 182 tweets réellement non-négatifs, 177 tweets réellement négatifs.

Interprétation : symétrique de la matrice positive — logique, puisque positive et negative sont dérivés du même score continu par un seuil unique (score > 0 / score < 0). Le modèle capture correctement 128 des 177 tweets négatifs (rappel 72 %), mais en rate 49, qui sont essentiellement les mêmes tweets que les 49 faux positifs de la classe positive vus sous un autre angle.

3. Précision, rappel, F1-score

Classe	Précision (classe cible)	Rappel (classe cible)	F1-score (classe cible)	Support
Positive	0.75	0.82	0.79	182

Négative	0.80	0.72	0.76	177
----------	------	------	------	-----

Accuracy globale : 77.4 % (359 tweets).

Interprétation : les deux classes sont évaluées à un niveau proche ($F1 \approx 0.76-0.79$), ce qui montre que le modèle n'est pas dégénéré (il ne prédit pas toujours la même classe). On observe cependant une asymétrie : le rappel est meilleur sur la classe positive (0.82) que sur la classe négative (0.72) — le modèle a tendance à pencher légèrement vers des prédictions positives en cas d'ambiguïté, ce qui explique la précision plus faible côté positif (0.75, à cause des faux positifs) et plus élevée côté négatif (0.80).

4. Analyse des performances : forces, faiblesses, biais

4.1 Forces

- Sur un jeu de validation **jamais vu à l'entraînement**, le modèle atteint 77.4 % d'accuracy avec seulement 50 000 exemples d'entraînement et une représentation TF-IDF + régression logistique — une base de référence raisonnable et non dégénérée (aucune classe n'est ignorée).
- Les cas lexicalement explicites (vocabulaire de sentiment clair : *love*, *hate*, *terrible*, *amazing*...) sont bien traités, ce qui est cohérent avec une approche sac-de-mots.

4.2 Faiblesses et biais identifiés (exemples réels tirés du jeu de validation)

a) Biais lié aux entités nommées / sujets d'actualité plutôt qu'au sentiment réel. Plusieurs faux positifs concernent des tweets d'actualité politique/économique au ton neutre ou factuel, mais mentionnant des entités (AIG, Obama, Cheney, Bush, North Korea) que le modèle associe à tort à un score positif :

[+0.62] "*Cheney and Bush are the real culprits - ...*" [+0.42] "*Obama Administration Must Stop Bonuses to AIG Ponzi Schemers ...*" [+0.27] "*Can we just go ahead and blow North Korea off the map already?*"

Le modèle apprend des corrélations lexicales de son corpus d'entraînement plutôt qu'une véritable compréhension du sentiment — un biais classique des modèles sac-de-mots.

b) Négation mal gérée. TF-IDF sur unigrammes ne capture pas l'inversion de sens apportée par une négation :

[-0.02] "*I current use the Nikon D90 and love it, but not as much as the Canon 40D/50D. I chose the D90 for the video feature. My mistake.*" (réel : positif, prédit : négatif)

c) Expressions familières / enthousiasme informel mal reconnu, probablement sous-représenté dans le vocabulaire appris :

[-0.66] "*#lebron best athlete of our generation...*" [-0.36] "*@wordwhizkid LeBron is a beast... nobody in the NBA comes even close.*"

d) Le modèle ne peut pas représenter un sentiment neutre ou mixte. Comme positive et negative proviennent du même score continu seuillé à 0, un tweet ne peut jamais être prédit "ni positif ni négatif" sauf dans le cas limite $\text{score} == 0$. Le jeu de validation Sentiment140 exclut d'ailleurs les tweets neutres (label 2) — le modèle n'a donc jamais été testé sur ce cas, alors que le schéma tweets (colonnes positive/negative indépendantes) le permettrait en théorie.

5. Recommandations d'amélioration

1. **Gérer les négations** : passer `TfidfVectorizer(ngram_range=(1, 2))` pour capturer des bigrammes comme *"not good"*, *"not as much"*, qui inversent le sens d'un unigramme isolé.
2. **Prétraitement adapté aux tweets** : normaliser les mots étirés (*loooove* → *love*), gérer les emojis/emoticons, hashtags et mentions, qui sont fréquents dans ce type de texte et actuellement traités comme des tokens bruts.
3. **Réduire le biais lié aux entités nommées** : envisager un nettoyage ou un poids réduit sur les noms propres/entités, ou enrichir le corpus d'entraînement pour diluer les corrélations spurieuses observées en §4.2.a.
4. **Passer à l'échelle** : le modèle actuel n'utilise que 50 000 tweets sur les 1.6M disponibles dans `Sentiment140`. Entraîner sur un volume plus important, avec une recherche d'hyperparamètres (régularisation `C`, `class_weight`), devrait réduire l'asymétrie de rappel observée en §3.
5. **Représenter les tweets neutres ou mixtes** : distinguer explicitement un score proche de 0 ("neutre") d'un score franchement positif ou négatif, plutôt que de forcer une classification binaire stricte à chaque tweet.
6. **Suivre la performance dans le temps** : comme le modèle est réentraîné automatiquement chaque semaine (`cronjob`, voir `CRONTAB_INSTRUCTIONS.md`), il est recommandé de relancer `evaluate.py` sur un jeu de validation fixe à chaque réentraînement pour détecter une éventuelle dérive de performance, plutôt que de ne l'exécuter qu'une fois.
7. **Élargir le jeu de validation** : 359 tweets donnent des métriques indicatives mais avec une marge d'incertitude non négligeable ; un jeu de validation plus grand donnerait des intervalles de confiance plus robustes pour le suivi qualité du modèle.